TUTORIAL VI

DISPLAYED TEXT

There are many instances in a document when we want to visually separate a portion of text from its surrounding material. One method of doing this is to typeset the distinguished text with added indentation. It is called *displaying*. LaTeX has various constructs for displaying text depending the nature of the displayed text.

VI.1. BORROWED WORDS

Quotations are often used in a document, either to add weight to our arguments by referring to a higher authority or because we find that we cannot improve on the way an idea has been expressed by someone else. If the quote is a one-liner, we can simply include it within double-quotes and be done with it (remember how to use quotes in TeX?) But if the quotation is several lines long, it is better to display it. Look at the following example:

Some mathematicians elevate the spirit of Mathematics to a kind of intellectual aesthetics. It is best voiced by Bertrand Russell in the following lines.

> The true spirit of delight, the exaltation, the sense of being more than man, which is the touchstone of the highest excellence, is to be found in Mathematics as surely as in poetry.... Real life is, to most men, a long second best, a perpetual compromise between the ideal and the possible; but the world of pure reason knows no compromise, no practical limitations, no barriers to the creative activity embodying in splendid edifices the passionate aspiration after the perfect, from which all great work springs.

Yes, to men like Russell, Mathematics is more of an art than science.

This was type set as shown below

```
Some mathematicians elevate the spirit of Mathematics  to a kind of
intellectual aesthetics. It is best voiced  by Bertrand Russell in the
following lines.
 \begin{quote}
   The true spirit of ..............................from which
   all great work springs.
 \end{quote}
```

Note that here we give instructions to TeX to typeset some material in a separate paragraph with additional indentation on either side and indicate the start and end of material requiring special treatment, by means of the commands

```
\begin{quote} ... \end{quote}
```

This is an example of what is known in LaTeX parlance as an *environment*. Environments are used to delimit passages requiring special typographic treatments and to give instructions to LaTeX on how to typeset it. The general form of an environment is of the form

```
\begin{name} ... \end{name}
```

where *name* is the name of the environment and signifies to LaTeX the type of typographic treatment required (deliberate attempt at a pun, that).

The quoted part in this example is a single paragraph. If the quotation runs into several paragraphs, we must use the `quotation` environment, by enclosing the quotation within `\begin{quotation}` and `\end{quotation}`. As usual, paragraphs are separated by blank lines while typing the source file.

VI.2. Poetry in typesetting

LaTeX can write poetry...well almost; if you write poems, TeX can nicely typeset it for you. (I have also heard some TeX wizards saying Knuth's code is sheer poetry!) Look at the passage below:

> Contrary to popular belief, limericks are not always ribald. Some of them contain mathematical concepts:
>
> A mathematician once confided
> That a Möbius band is one sided
> You'll get quite a laugh
> If you cut it in half
> For it stays in one piece when divided
>
> There is an extension of this to Klein's bottle also.

This was typeset as follows:

```
Contrary to popular belief, ...  tried their hands at it:
\begin{verse}
  A mathematician confided\\
  A M\"obius band is one sided\\
  You'll get quite a laugh\\
  If you cut it in half\\
  For it stays in one piece when divided
\end{verse}
There is an extension of this to Klein's bottle also.
```

Note that line breaks are forced by the symbol `\\`. Different stanzas are separated in the input by one (or more) blank lines. If you do not want TeX to start a new page at a particular line break (if you want to keep rhyming couplets together in one page, for example), then use `\\*` instead of plain `\\`. Again, if you want more space between lines than what LaTeX deems fit, then use `\\` with an optional *length* as in `\\[5pt]` which adds an *extra* vertical space of 5 points between the lines. You can also type `\\*[5pt]`, whose intention should be obvious by now.

VI.3. Making lists

Lists are needed to keep some semblance of order in a chaotic world and LaTeX helps us to typeset them nicely. Also, there are different kinds of lists available by default and if

none of them suits your need, there are facilities to tweak these or even design your own. Let us first have a look at the types of lists LaTeX provides.

VI.3.1. Saying it with bullets

The `itemize` environment gives us a bullet-list. For example it produces something like this:

One should keep the following in mind when using TeX

- TeX is a typesetting language and not a word processor
- TeX is a program and and not an application
- Theres is no meaning in comparing TeX to a word processor, since the design purposes are different

Being a program, TeX offers a high degree of flexibility.

The input which produces this is given below:

```
One should keep the following in mind when using \TeX
\begin{itemize}
\item \TeX\ is a typesetting language and not a word processor
\item \TeX\ is a program and and not an application
\item Theres is no meaning in comparing \teX\ to a word processor, since the design
  purposes are different
\end{itemize}
Being a program, \TeX\ offers a high degree of flexibility.
```

The `\begin{itemize} ... \end{itemize}` pair signifies we want a bullet-list of the enclosed material. Each item of the list is specified by (what else?) an `\item` command.

We can have lists within lists. For example:

One should keep the following in mind when using TeX

- TeX is a typesetting language and not a word processor
- TeX is a program and and not an application
- Theres is no meaning in comparing TeX to a word processor, since the design purposes are different
- TeX is the natural choice in one of these situations
  - If we want to typeset a document containing lot of Mathematics
  - If we want our typed document to look beautiful

Being a program, TeX offers a high degree of flexibility.

It is produced by the input below:

```
One should keep the following in mind when using \TeX
\begin{itemize}
\item \TeX\ is a typesetting language and not a  word processor
\item \TeX\ is a program and and not an application
\item Theres is no meaning in comparing \TeX\ to a word processor, since the design
      purposes are different
\item \TeX\ is the natural choice in one of these situations
  \begin{itemize}
  \item If we want to typeset a document containing lot of Mathematics
```

```
  \item If we want our typed document to look beautiful
  \end{itemize}
\end{itemize}
Being a program, \TeX\ offers a high degree of flexibility.
```

The `itemize` environment supports four levels of nesting. The full list of *labels* for the items ('bullets' for the first level, 'dashes' for the second and so on) is as shown below

- The first item in the first level
- the second item in the first level
  - The first item in the second level
  - the second item in the second level
    - The first item in the third level
    - the second item in the third level
      - The first item in the fourth level
      - the second item in the fourth level

Not satisfied with these default labels? How about this one?

- ▹ First item of a new list
- ▹ Second item

It was produced by the following input:

```
{\renewcommand{\labelitemi}{$\triangleright$}
\begin{itemize}
\item First item of a new list
\item Second item
\end{itemize}}
```

Several things need explanation here. First note that the *first level labels* of the `itemize` environment are produced by the (internal and so invisible to the user) command `\labelitemi` and by default, this is set as `\textbullet` to produce the default 'bullets'. What we do here by issuing the `\renewcommand` is to override this by a choice of our own namely `$\triangleright$` which produces the little triangles in the above list. Why the braces `{` and `}` (did you notice them?) enclosing the whole input? They make the effect of the `\renewcommand` *local* in the sense that this change of labels is only for this specific list. Which means the next time we use an `itemize` environment, the labels revert back to the original 'bullets'. If we want the labels to be changed in the *entire document*, then remove the braces.

What if we want to change the second level labels? No problem, just change the `\labelitemii` command, using a symbol of our choice. The third and fourth level labels are set by the commands (can you guess?) `\labelitemiii` and `\labelitemiv`. Look at the following example.

- ☛ The first item in the first level
- ☛ the second item in the first level
  - ☞ The first item in the second level
  - ☞ the second item in the second level
    - ✌ The first item in the third level
    - ✌ the second item in the third level
      - ✍ The first item in the fourth level
      - ✍ the second item in the fourth level

Here the labels are chosen from the PostScript ZapfDingbats font. We will have to use the package pifont, by including the line `\usepackage{pifont}` in our document preamble to access them. The source of the above output is

```
\renewcommand{\labelitemi}{\ding{42}}
\renewcommand{\labelitemii}{\ding{43}}
\renewcommand{\labelitemiii}{\ding{44}}
\renewcommand{\labelitemiv}{\ding{45}}
 \begin{itemize}
  \item The first item in the first level
  \item the second item in the first level
   \begin{itemize}
    \item The first item in the second  level
    \item the second item in the second level
     \begin{itemize}
      \item The first item in the third  level
      \item the second item in the third level
       \begin{itemize}
        \item The first item in the fourth  level
        \item the second item in the fourth level
       \end{itemize}
     \end{itemize}
   \end{itemize}
  \end{itemize}
 \end{itemize}}
```

VI.4. WHEN ORDER MATTERS

When the *order* of the items in a list is important, we need a list which specifies this order. For example, consider this

The three basic steps in producing a printed document using LaTeX are as follows

1. Prepare a source file with the extension `tex`
2. Compile it with LaTeX to produce a `dvi` file
3. Print the document using a `dvi` driver

Such a numbered list is produced by the `enumerate` environment in LaTeX. The above list was produced by the following source.

```
\begin{enumerate}
\item prepare a source file with the extension "tex"
```

```
\item Compile it with \LaTeX to produce a "dvi" file
\item Print the document using a "dvi" driver
\end{enumerate}
```

As in the case of `itemize` environment, here also four levels of nesting are supports. The example below shows the labels used for different levels.

1. The first item in the first level
2. the second item in the first level
   (a) The first item in the second level
   (b) the second item in the second level
      i. The first item in the third level
      ii. the second item in the third level
         A. The first item in the fourth level
         B. the second item in the fourth level

How about customizing the labels? Here there is an additional complication in that the labels for items in the same level must follow a sequence (such as 1,2,3,... for the first level, (a), (b), (c),... for the second and so on, by default). There *is* a method for doing it, but it will take us into somewhat deeper waters. Fortunately, there is a package enumerate by David Carlisle, which makes it easy. So if we want

The three basic steps in producing a printed document using LaTeX are as follows:

Step 1. Prepare a source file with the extension `tex`
Step 2. Compile it with LaTeX to produce a `dvi` file
   i. Use a previewer (such as `xdvi` on X Window System) to view the output
   ii. Edit the source if needed
   iii. Recompile
Step 3. Print the document using a `dvi` driver (such as `dvips`)

just type the input as follows

```
The three basic steps in  producing a printed document
using \LaTeX\ are as follows:
\begin{enumerate}[\hspace{0.5cm}Step 1.]
\item Prepare a source file with the extension "tex"
\item Compile it with \LaTeX to produce a "dvi" file
  \begin{enumerate}[i.]
  \item Use a previewer (such as "xdvi" on
    \textsf{X Window System}) to view the output
  \item Edit the source if needed
  \item Recompile
  \end{enumerate}
\item Print the document using a "dvi" driver
  (such as "dvips")
\end{enumerate}
```

As you can see, the labels Step 1, Step 2 and Step 3 are produced by the *optional* argument `Step 1` within square brackets immediately following the first `\begin{enumerate}` command and the labels i, ii, iii for the second level enumeration are produced by the optional `[i]` following the second `\begin{enumerate}`. So, what is `\hspace{0.5cm}` doing in the first optional argument? It is to provide an indentation at the left margin of the first level items, which the **enumerate** environment does not produce by default.

We can add further embellishments. For example, if we want the labels in the first level of the above example to be in boldface, just change the optional argument `[\hspace{0.5cm} Step 1]` to `[\hspace{0.5cm}\bfseries Step 1]`. This produces:

The three basic steps in producing a printed document using LaTeX are as follows:

Step 1 Prepare a source file with the extension `tex`
Step 2 Compile it with LaTeX to produce a `dvi` file
  (a) Use a previewer (such as `xdvi` on X Window System) to view the output
  (b) Edit the source if needed
  (c) Recompile
Step 3 Print the document using a `dvi` driver (such as `dvips`)

Some care must be taken when we give options like this. Suppose we want to produce something like this

Addition of numbers satisfies the following conditions:

(A1) It is commutative
(A2) It is associative
(A3) There is an additive identity
(A4) Each number has an additive inverse

If we give the option `[\hspace{1cm}(A1)]` as in

```
Addition of numbers satisfies the following conditions:
  \begin{enumerate}[\hspace{1cm}(A1)]
  \item It is commutative
  \item It is associative
  \item There is an additive identity
  \item Each  number has an additive inverse
  \end{enumerate}
```

Then we get the (somewhat surprising) output

(11) It is commutative
(22) It is associative
(33) There is an additive identity
(44) Each number has an additive inverse

What happened? In the enumerate package, the option `[A]` signifies that we want the labels to be named in the sequence A, B, C,...,Z (the upper case Roman alphabet) and the option `[1]` signifies we want them as 1,2,3,... (the Arabic numerals). Other signifiers are `[a]` for lowercase Roman letters, `[I]` for uppercase Roman numerals and `[i]` for lowercase Roman numerals. So, if we use any one of these in the optional argument with some other purpose in mind, then *enclose it in braces*. Thus the correct input to generate the above example is

```
Addition of numbers satisfies the following conditions
 \begin{enumerate}[\hspace{1cm}({A}1)]
 \item It is commutative
 \item It is associative
 \item There is an additive identity
```

```
\item Each  number has an additive inverse
\end{enumerate}
```

with braces surrounding the A. (The mystery is not over, is it? How come we got 11, 22,... in the above example and not A1, B2,...? Work it out yourselves!)

VI.5. DESCRIPTIONS AND DEFINITIONS

There is a third type of list available off-the-shelf in LaTeX which is used in typesetting lists like this

Let us take stock of what we have learnt

TeX A typesetting program

Emacs A text editor and also

a programming environment

a mailer

and a lot else besides

AbiWord A word processor

This is produced by the `description` environment as shown below:

```
Let us take stock of what we have learnt
  \begin{description}
  \item[\TeX] A typesetting program
  \item[Emacs] A text editor and also
    \begin{description}
    \item a programming environment
    \item a mailer
    \item and a lot else besides
    \end{description}
  \item[AbiWord] A word processor
  \end{description}
```

Note that this environment does not produce on its own any labels for the various items, but only produces as labels, whatever we give inside square brackets immediately after each `\item`. By default, the labels are typeset in boldface roman. Also, there is no indentation for the first level. As with the other list environments, these can be changed to suit your taste. For example, suppose we want labels to be typeset in sans-serif roman and also want an indentation even for the first level. The code below will do the trick (remember why we include the whole input within braces?):

```
\renewcommand{\descriptionlabel}[1]{\hspace{1cm}\textsf{#1}}
Let us take stock of what we have learnt
 \begin{description}
  \item[\TeX] A typesetting program
  \item[Emacs] A text editor and also
    \begin{description}
    \item a programming environment
    \item and a lot else besides
    \end{description}
  \item[AbiWord] A word processor
 \end{description}
```

and we get the output

Let us take stock of what we have learnt

TEX A typesetting program

Emacs A text editor and also

a programming environment

and a lot else besides

AbiWord A word processor

Now is perhaps the time to talk about a general feature of all the three list environments we have seen. In any of these, we can override the default labels (if any) produced by the environment by something of our own by including it within square brackets immediately after the `\item`. Thus the input

```
The real number $l$ is the least upper bound of the
set $A$ if it satisfies the following conditions
  \begin{enumerate}
  \item[(1)] $l$ is an upper bound of $A$
  \item[(2)] if $u$ is an upper bound of $A$, then $l\le u$
  \end{enumerate}
The second condition is equivalent to
  \begin{enumerate}
  \item[(2)$'$] If $a<l$, then $a$ is not an upper bound of $A$.
  \end{enumerate}
```

produces

The real number l is the least upper bound of the set A if it satisfies the following conditions

(1) l is an upper bound of A

(2) if u is an upper bound of A, then $l \le u$

The second condition is equivalent to

(2)′ If $a < l$, then a is not an upper bound of A.

This feature sometimes produces unexpected results. For example, if you type

```
Let's review the notation
 \begin{itemize}
 \item (0,1) is an \emph{open} interval
 \item [0,1] is a \emph{closed} interval
 \end{itemize}
```

you will get

Let's review the notation

- (0,1) is an *open* interval

0,1 is a *closed* interval

What happened? The `0,1` within *square brackets* in the second item is interpreted by LaTeX as the *optional label* for this item. The correct way to typeset this is

```
Let's review the notation
 \begin{itemize}
 \item $(0,1)$ is an \emph{open} interval
 \item $[0,1]$ is a \emph{closed} interval
 \end{itemize}
```

which produces

Let's review the notation

- $(0,1)$ is an *open* interval
- $[0,1]$ is a *closed* interval

So, why the dollars around `(0,1)` also? Since `(0,1)` and `[0,1]` are *mathematical entities*, the correct way to typeset them is to include them within braces in the input, even when there is no trouble such as with `\item` as seen above. (By the way, do you notice any difference between (o,ı) produced by the input `(0,1)` and $(0,1)$ produced by `$(0,1)$`?)

In addition to all these tweaks, there is also provision in LaTeX to design your own 'custom' lists. But that is another story.